

Introduction & Context

Ask students:

“When you move your head in VR, how does the system update the scene instantly?”

Explain:

VR is not just graphics — it is a **real-time interactive system** made of:

- Input
- Processing
- Rendering
- Output

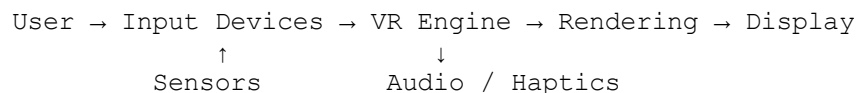
Today we study how all these are organized.



PART 1: VR Systems Overview

Basic VR System Components

Draw a simple block diagram on board:



Core Components of a VR System:

1 Input Subsystem

- Head tracking (IMU, cameras)
 - Hand controllers
 - Gloves
 - Voice input
-

2 Processing / VR Engine

- Scene management
 - Physics engine
 - Collision detection
 - Interaction logic
 - AI behaviors
-

3 Rendering System

- Stereo rendering (left + right eye)
 - Lighting
 - Shading
 - Frame buffering
-

4 Output Subsystem

- HMD display
 - Spatial audio
 - Haptic feedback
-

Key Idea to Emphasize:

VR architecture integrates **graphics + physics + input + feedback** in one synchronized loop.



PART 2: Data Flow in VR Applications

Understanding the VR Interaction Loop

Explain the **VR Frame Loop**:

Every frame (~90 times per second):

1. Capture input (head movement, controller position)
2. Update world state
3. Run physics & interactions
4. Render scene
5. Display to headset
6. Provide feedback (audio/haptics)

Write on board:

Input → Update → Render → Output → Repeat

Important Concept: Real-Time Constraint

VR typically requires:

- 60–120 FPS
- Motion-to-photon latency < 20 ms

Explain:

If data flow is delayed → motion sickness.

Detailed Data Flow Explanation

Break it into layers:

Layer 1: Sensor Data Collection

- IMU sends rotation data
 - Cameras send position data
-

Layer 2: Tracking Module

- Sensor fusion algorithm
 - Calculates 6DoF (position + orientation)
-

Layer 3: Application Logic

- Updates object positions
 - Detects collisions
 - Executes scripts
-

Layer 4: Rendering Engine

- Generates left-eye image
 - Generates right-eye image
-

Layer 5: Output

- Frame sent to display
 - Audio engine plays 3D sound
 - Haptic motor vibrates
-

What is LOD?

LOD (Level of Detail) is a technique used in computer graphics where

Different versions of the same 3D object are used depending on how far it is from the camera.

In simple words:

Objects far away use simpler models.

Objects close to the viewer use detailed models.

Why Do We Need LOD in VR?

VR must run at:

- 60–120 FPS
- Very low latency

If every object always used high-detail models:

- GPU overload
- Frame drops
- Motion sickness

LOD helps maintain performance.



Example

Imagine a VR temple scene:

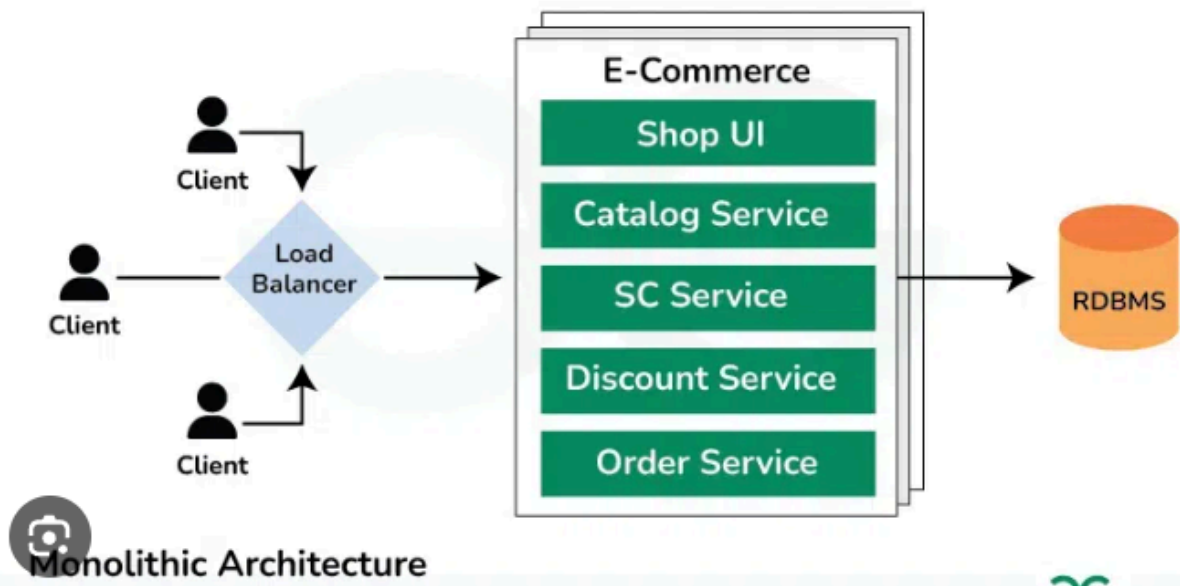
Distance from User	Model Used
1 meter (very close)	High polygon detailed statue
10 meters	Medium polygon version
50 meters	Very low polygon version

Software Architecture Models in VR

two common models:

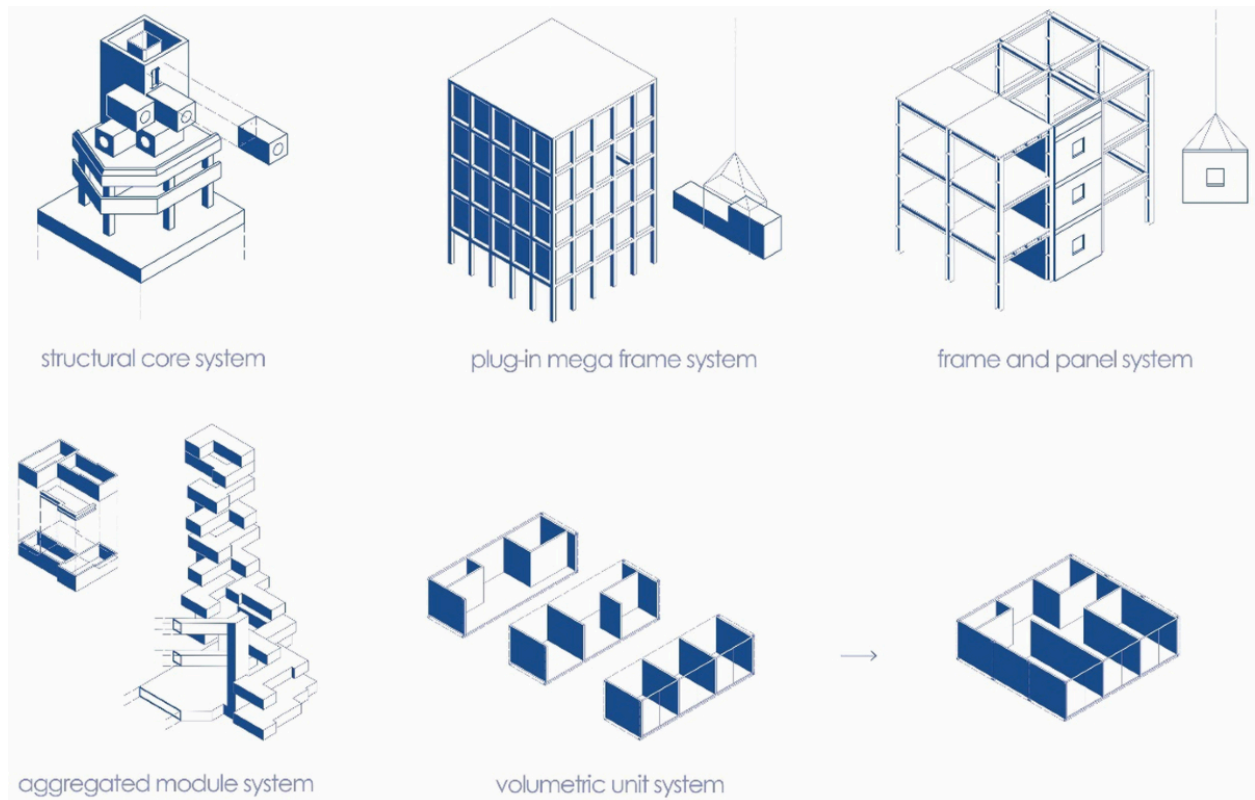
1 Monolithic Architecture

- Everything inside one engine
- Simpler
- Used in games



2 Modular Architecture

- Separate modules:
 - o Tracking module
 - o Rendering module
 - o Physics module
 - o Interaction module
- Used in complex industrial VR



Why Modern VR Systems Prefer Modular Architecture for Scalability

◆ First: What is Modular Architecture?

Modular architecture means:

The VR system is divided into independent components (modules), each responsible for a specific function.

Instead of one big program doing everything, we separate it into:

- Tracking Module
- Rendering Module
- Physics Module
- Interaction Module

- Audio Module
- Networking Module

Each module works independently but communicates with others.

◆ What is Scalability?

Scalability means:

The system can grow, upgrade, or handle more complexity without breaking or being rewritten completely.

Example:

- Adding multiplayer support
 - Supporting a new VR headset
 - Adding AI avatars
 - Increasing scene complexity
-

Why Modular Architecture is Better for Scalability

1 Easy to Upgrade Individual Components

If rendering improves (e.g., new ray tracing tech):

You only update:

👉 Rendering Module

You don't rewrite:

- ✗ Tracking
- ✗ Physics
- ✗ Interaction

This makes upgrades simpler and cheaper.

2 Supports Multiple Devices

Modern VR systems must support:

- Meta Quest
- HTC Vive
- HoloLens
- OpenXR devices

With modular design:

- Device-specific code is inside **Hardware Interface Module**
- Core app logic remains same

So:

One app → Multiple devices → Easy expansion

3 Parallel Development (Team Advantage)

In industry:

- One team works on rendering
- One team works on physics
- One team works on networking
- One team works on UI

Because modules are separate, teams don't interfere with each other.

This increases development speed.

4 Better Performance Optimization

If FPS drops:

- You analyze rendering module
- Optimize shaders
- Adjust LOD

No need to touch entire system.

Fault Isolation

If tracking fails:

- Only tracking module affected
- Rest of system remains stable

In monolithic architecture:

One bug → Whole system crashes.

Example Comparison

Monolithic VR System

Everything inside one large codebase:

Input + Rendering + Physics + Audio + AI all mixed

Hard to:

- Debug
 - Upgrade
 - Scale
 - Maintain
-

Modular VR System

Input Module
Tracking Module
Physics Module
Rendering Module
Audio Module
Networking Module

Each module:

- Replaceable

- Maintainable
 - Scalable
-

◆ Real Industry Example

Unity XR Architecture

- XR Plugin (device interface)
- Rendering pipeline
- Physics engine
- Script system
- Audio system

Each is separate but integrated.

Real Example (Unity VR Application)

Explain practical workflow:

1. Unity scene loads
2. XR plugin activates
3. SDK connects to headset
4. Tracking data updates camera
5. Scene renders stereoscopic view
6. Frame sent to device

This connects software architecture to real development.

Recap + Quick Questions

Recap Key Points:

- VR software architecture connects input, processing, rendering, and output
- VR operates in a continuous real-time loop
- Low latency is critical
- Modular design improves scalability

1 Short Answer (2–3 Marks)

- **Define VR Software Architecture.**
Explain its main purpose in a VR system.
- **Explain the main components of a VR system architecture.**
Describe the role of input, processing, rendering, and output modules.
- **Explain the data flow in a VR application.**
Illustrate the VR interaction loop with a neat diagram.
- **Discuss modular architecture in VR systems. Why is it preferred over monolithic architecture?**
- When a user moves their head in a VR headset, explain step-by-step how the system processes this input and updates the display.